Spec-Driven Development Suite

A reusable workflow for AI-assisted software delivery

Connected requirements, implementation and verification

Release 0.4.3

The delivery problem

01 **Scope needs a shared definition**

A request needs observable acceptance checks before completion can be assessed.

02 **Changes affect more than code**

A changed requirement can affect the design, task plan and earlier test results.

03 **Handover needs durable context**

The next developer or tester needs the decisions, remaining work and actual checks.

One workflow, twelve focused skills

Delivery stage	Skills
Define the feature	Discovery, requirements and technical design
Deliver the change	Task planning, implementation and acceptance verification
Maintain continuity	Review, change analysis and resumption
Fit the project	Workflow coordination, project guidance and event checks

Use the full workflow or request an individual stage.

A feature with reviewable evidence

Example: a maximum-length option for a slug helper

01 **Intended behavior**

Requirements define valid bounds, existing callers and trailing separators.

02 **Implementation and meaningful tests**

The supplied source and five tests exercise the agreed behavior.

03 **Current completion evidence**

Recorded output links checks to the feature. Changed inputs make evidence stale.

Package and validation

Included today	Recorded checks
Twelve skills and local tools	95 tool checks passed, 4 environment-related skips
Automatic skill selection	36/36 expected outcomes in one model environment
Codex and Claude Code setup	User and project installation layouts checked
Versioned distribution	File manifest, checksum and recovery tools

Actual Claude model behavior remains outside the recorded tests.

Detailed methods and versions: docs/RELEASE-REPORT.md

First use in a team project

01 **Install the package**

Choose the coding assistant and a project or user installation.

02 **Request a suitable feature or stage**

Set the intended outcome and any review boundaries in normal language.

03 **Assess the work through its records**

Review the code, recorded checks and remaining gaps with the team.

Next step: choose a feature and run the included demonstration.